MNIST Handwritten Digit Classification

Complete Architecture Walkthrough & Function-by-Function Reference Guide

Target Notebook: project.ipynb
Frameworks: TensorFlow 2.x, OpenCV, NumPy, Seaborn
Generated: October 2026

Executive Summary: This project explores the end-to-end development, training, evaluation, and practical deployment of deep learning models for classifying handwritten digits (0–9) using the benchmark **MNIST** dataset (70,000 images). The notebook implements a dual-architecture comparison: a baseline **Dense Artificial Neural Network (ANN)** achieving **97.84% test accuracy**, and a spatial-aware **Convolutional Neural Network (CNN)** achieving a superior **99.21% test accuracy**. Furthermore, the project creates an end-to-end **Computer Vision inference pipeline** using OpenCV to preprocess real-world external test images (such as 5.png and 4.png) to perform real-time interactive predictions.

1. Project Architecture & Workflow Overview

The project follows a standard machine learning lifecycle structured into 6 sequential phases:

- **Phase 1: Data Acquisition & Exploratory Analysis:** Loading the 70,000 MNIST digit images, inspecting array shapes, data types, pixel ranges, and visualizing ground-truth digit samples.
- **Phase 2: Data Normalization / Scaling:** Applying Min-Max normalization by dividing raw 8-bit pixel values (0–255) by 255.0 to map input features into the float range [0.0, 1.0].
- **Phase 3: Baseline Neural Network (Dense ANN):** Building a fully-connected feed-forward network with Flatten, two hidden Dense layers (128 & 64 units with ReLU), and a 10-unit Softmax output layer.
- **Phase 4: Diagnostics & Error Analysis:** Generating probability distributions via model.predict(), computing argmax predictions, and plotting a 10x10 Confusion Matrix heatmap to detect digit confusion.
- **Phase 5: Advanced Spatial Model (Deep CNN):** Constructing a 2D Convolutional Neural Network with Conv2D, MaxPooling2D, and Dropout(0.25) to retain spatial relationships and curb overfitting.
- **Phase 6: Real-World Predictive System:** Developing an image-processing pipeline using OpenCV (cv2) to load external camera/scan images, convert RGB to Grayscale, resize to 28x28, normalize, and predict.

High-Level Model Comparison (ANN vs CNN)

Metric / Feature	Baseline ANN (model)	Deep CNN (model_cnn)	Advantage / Impact
Input Transformation	Flatten(28, 28) → 784-dim vector	Reshape((28,28,1)) (Keeps 2D grid)	CNN preserves 2D spatial pixel neighborhoods
Feature Extraction	None (learns global weights directly)	2x Conv2D (32, 64) + 2x MaxPool (2x2)	Extracts translation-invariant edges & loops
Regularization	None (susceptible to overfitting)	Dropout(0.25) before Dense layer	Prevents co-adaptation of features
Training Accuracy	~99.30% (after 10 epochs)	~99.46% (with 10% validation split)	Both models train smoothly with Adam
Test Accuracy (10,000)	**97.84%**	**99.21%**	**+1.37% gain** (~63% test error reduction)
External Image Robustness	Lower (failed on 5.png)	High (predicts correctly via local patterns)	Crucial for real-world digit deployment

2. Comprehensive Function & Method Reference

Below is an exhaustive breakdown of every key library function, method, layer, and API utilized in project.ipynb, detailing exactly what work it performed and why it was required.

Library / Module	Function / Method	What Work It Did in This Project	Technical Purpose & Why Used
tensorflow.keras.datasets	`mnist.load_data()`	Downloaded and unpacked the 70,000 MNIST images into 60k training samples and 10k test samples.	Standard benchmark loader; automatically splits inputs (X) and ground-truth targets (y).
tensorflow.keras	`tf.keras.Sequential([...])`	Instantiated the model containers for both the baseline ANN (model) and the CNN (model_cnn).	Defines a linear stack of neural network layers where tensor data flows sequentially from input to output.
tf.keras.layers	`Flatten(input_shape=(28, 28))`	Transformed each 2D 28x28 pixel matrix into a flat 1D vector of 784 elements in the baseline ANN.	Dense layers require 1D vector inputs; Flatten reshapes without altering batch dimensions.
tf.keras.layers	`Dense(units, activation)`	Created fully connected layers: 128 & 64 units (ReLU) for feature representation, and 10 units (Softmax) for output.	Performs matrix multiplication (W·X + b). ReLU adds non-linearity; Softmax outputs normalized probabilities.
tf.keras.layers	`Reshape((28, 28, 1))`	Converted 2D grayscale matrices (28, 28) into explicit 3D image tensors (28, 28, 1) at the entrance of the CNN.	Conv2D operations require explicit channel dimensions (Height x Width x Channels).
tf.keras.layers	`Conv2D(filters, kernel, activation)`	Block 1 used 32 filters (3x3); Block 2 used 64 filters (3x3) to convolve over image features.	Extracts spatial visual features (edges, curves, loops) while maintaining positional relationship of pixels.
tf.keras.layers	`MaxPooling2D(pool_size=(2,2))`	Downsampled feature map dimensions by half after each Conv2D block (from 28x28 to 13x13 to 5x5).	Reduces spatial footprint, cuts computational cost, and grants spatial translation invariance.
tf.keras.layers	`Dropout(0.25)`	Randomly deactivated 25% of neurons during each CNN training pass.	Acts as an effective regularizer, preventing co-adaptation of neurons and reducing overfitting.
tf.keras.Model	`model.compile(optimizer, loss, metrics)`	Configured both networks with adam optimizer, sparse_categorical_crossentropy, and accuracy.	Compiles the computational graph. Sparse cross-entropy handles integer labels without one-hot encoding.
tf.keras.Model	`model.fit(X, y, epochs, ...)`	Executed iterative training: ANN trained for 10 epochs; CNN trained for 10 epochs with batch size 64 and 10% validation.	Performs forward pass, backpropagation, gradient calculation, and weight optimization.
tf.keras.Model	`model.evaluate(X_test, y_test)`	Evaluated test loss and accuracy across the 10,000 unseen benchmark samples.	Measures unbiased generalization performance on out-of-sample data.
tf.keras.Model	`model.predict(X)`	Generated raw 10-dimensional Softmax probability vectors for test sets and custom single images.	Executes inference mode forward-pass without backpropagation.
tf.keras.Model	`model_cnn.summary()`	Printed layer names, output tensor shapes, and total trainable parameter count for the CNN.	Essential for verifying tensor dimensionality and calculating network parameter complexity.
tensorflow.math	`confusion_matrix(labels, preds)`	Computed a 10x10 contingency matrix comparing actual test labels against predicted labels.	Pinpoints specific digit misclassifications (e.g. false positives between 4 and 9 or 3 and 5).
cv2 (OpenCV)	`cv2.imread(path)`	Loaded external custom images (e.g. 5.png and user-prompted paths) into NumPy arrays.	Standard high-performance computer vision image loader (loads in BGR format by default).
cv2 (OpenCV)	`cv2.cvtColor(src, code)`	Converted BGR images to RGB (for Matplotlib display) and RGB to Grayscale (for model input).	MNIST models expect single-channel grayscale inputs; OpenCV loads 3-channel color by default.
cv2 (OpenCV)	`cv2.resize(src, (28, 28))`	Rescaled custom external images of arbitrary resolution down to exactly 28x28 pixels.	Aligns spatial resolution with the exact input dimensions expected by the trained neural network.
numpy (np)	`np.unique(array)`	Inspected target label arrays (y_train, y_test) to verify distinct classes [0..9].	Confirms all 10 integer digits are present without unexpected class values.
numpy (np)	`np.argmax(a, axis)`	Mapped 10-class probability vectors into single integer class predictions (0 through 9).	Finds the index corresponding to maximum confidence output from Softmax.

Library / Module	Function / Method	What Work It Did in This Project	Technical Purpose & Why Used
numpy (np)	`np.reshape(a, [1, 28, 28])`	Added a batch dimension of size 1 to single preprocessed test images before feeding into predict().	Keras inference expects batch-shaped tensors: (Batch_Size, Height, Width).
numpy (np)	`array / 255.0`	Scaled pixel values from integer range [0, 255] down to floating point [0.0, 1.0].	Min-Max normalization speeds up gradient descent convergence and prevents exploding gradients.
matplotlib.pyplot	`plt.imshow(img)`	Visualized digit images from MNIST and rendered preprocessed external input images.	Renders 2D and 3D numeric arrays as graphical color or grayscale images.
matplotlib.pyplot	`plt.axis('off')`	Removed coordinate axes, grid ticks, and borders when displaying input digit images.	Creates clean visual presentations of image data without distracting coordinate scales.
matplotlib.pyplot	`plt.figure(figsize=(15,7))`	Configured wide display dimensions for rendering the large 10x10 confusion matrix.	Ensures matrix cell values and class tick labels remain legible without overlap.
seaborn (sns)	`sns.heatmap(data, annot, ...)`	Plotted the 10x10 confusion matrix with integer counts (fmt='d') and blue color intensity.	Provides clear visual perception of classification density along the diagonal and off-diagonal errors.
Python Built-in	`input('Prompt text')`	Paused cell execution to prompt user for an image file path dynamically in the final cell.	Enables interactive runtime prediction on arbitrary user-provided image files.
Python Built-in	`list.append() / int()`	Collected argmax predictions across all 10,000 test cases into y_pred_labels.	Constructs a clean integer list of predicted labels for confusion matrix consumption.

3. Detailed Phase-by-Phase Technical Walkthrough

Phase 1 & 2: Dataset Loading, Exploration & Normalization

The project begins by ingesting the MNIST benchmark via datasets.mnist.load_data(). The dataset contains 60,000 training images and 10,000 testing images of dimension 28x28 pixels. Exploration using np.unique() verifies that the target classes are balanced integers from 0 to 9. Because raw 8-bit image pixels range from 0 to 255, the notebook performs feature scaling:
X_train = X_train / 255.0 and X_test = X_test / 255.0.
Why scaling is essential: Unnormalized inputs with large numerical values cause severe gradient variance, slowing down backpropagation and potentially causing exploding gradients. Scaling to [0.0, 1.0] ensures stable, smooth optimization for the Adam optimizer.

Phase 3: Baseline Neural Network (Dense ANN)

The first architecture built is a standard Multi-Layer Perceptron (ANN):
• **Flatten Layer:** Unrolls the (28, 28) matrix into a 784-element feature vector.
• **Dense Hidden Layer 1:** 128 neurons with ReLU activation (introduces non-linear decision boundaries).
• **Dense Hidden Layer 2:** 64 neurons with ReLU activation (extracts condensed hierarchical representations).
• **Output Layer:** 10 neurons with Softmax activation (outputs probability distribution summing to 1.0).
• **Loss & Optimizer:** sparse_categorical_crossentropy (avoids memory overhead of one-hot encoded vectors) paired with Adam (adaptive learning rates per parameter).
Result: Achieved **97.84% test accuracy** across 10 epochs.

Phase 4: Diagnostics, Confusion Matrix & ANN Custom Image Test

To thoroughly evaluate the ANN, predictions on all 10,000 test images are obtained via model.predict(X_test). Because Softmax outputs 10 probabilities per image, np.argmax() extracts the predicted class label. A 10x10 Confusion Matrix is generated using tf.math.confusion_matrix() and plotted via sns.heatmap(). This reveals that while overall accuracy is high, the ANN occasionally confuses visually similar digits (e.g. 4 vs 9, 3 vs 5, and 7 vs 2).
When tested on an external custom image (5.png), the ANN predicted class **7** instead of 5. This highlights the core flaw of standard dense ANNs: flattening destroys 2D spatial locality and spatial invariance, making the model hypersensitive to minor line thickness, stroke positioning, and background contrast.

Phase 5: Deep Convolutional Neural Network (CNN)

To resolve the limitations of the ANN, the project constructs a dedicated Convolutional Neural Network (CNN):
• **Input Reshape:** Explicitly forms a (28, 28, 1) 3D tensor representing (Height, Width, Channel).
• **Conv2D Block 1:** 32 filters of size (3x3) + ReLU. Learns local spatial filters such as horizontal/vertical edges and curves.
• **MaxPooling2D Block 1:** (2x2) pool window. Downsamples feature maps from 28x28 to 13x13, retaining the strongest feature activations.
• **Conv2D Block 2:** 64 filters of size (3x3) + ReLU. Synthesizes low-level features into complex digit components (loops, intersections).
• **MaxPooling2D Block 2:** Downsamples feature maps from 13x13 to 5x5.
• **Flatten & Regularization:** Flattens 5x5x64 (1,600 values) followed by Dropout(0.25) to inhibit overfitting.
• **Dense Classification:** 64-neuron Dense layer + 10-neuron Softmax classification head.
• **Training & Performance:** Trained with batch size 64 and a 10% validation split. Achieved **99.21% test accuracy** on test data, decisively outperforming the ANN.

Phase 6: Interactive Predictive System for Real-World Custom Images

The final section of the notebook creates an interactive inference system using OpenCV to classify user-supplied digit images:
1. **Input Acquisition:** Prompts user for image path via input().
2. **Image Reading:** cv2.imread() reads the image file.
3. **Display:** Converts BGR to RGB via cv2.cvtColor() and renders using plt.imshow().
4. **Color Channel Conversion:** Converts RGB to single-channel Grayscale via cv2.COLOR_RGB2GRAY.
5. **Spatial Resizing:** cv2.resize(grayscale, (28, 28)) rescales image to exact model dimensions.
6. **Normalization:** Divides pixel values by 255.0.
7. **Batch Dimension Injection:** np.reshape(input_scaled, [1, 28, 28]) creates tensor shape (1, 28, 28).
8. **Inference:** model_cnn.predict() outputs probabilities; np.argmax() prints the predicted digit.

4. Key Takeaways & Practical Recommendations

- **Why CNN Outperformed ANN:** The baseline ANN achieved 97.84% by treating the 784 pixels independently, losing all spatial structure. The CNN achieved 99.21% because convolution filters preserve local 2D pixel neighborhoods, enabling translation-invariant stroke and curve detection.
- **Background Inversion Caution:** MNIST images feature white strokes on a solid black background (pixel values near 0 for background, near 255 for stroke). Real-world photos or scanned documents typically feature black ink on white paper. If an inverted image is fed into the system, OpenCV preprocessing should include cv2.bitwise_not() to invert colors before feeding to the model.
- **Aspect Ratio Preservation:** Directly calling cv2.resize() on non-square images can distort stroke aspect ratios. For production systems, it is recommended to pad rectangular images to a square aspect ratio before resizing to 28x28.
- **Model Persistence Recommendation:** Currently, the notebook trains models in memory. To avoid retraining across sessions, the trained CNN should be saved to disk using model_cnn.save('mnist_cnn_model.keras') and reloaded with tf.keras.models.load_model().